生パケットベースの畳み込みニューラルネットワークとメタ学習を用いた軽量なDDoS攻撃検知フレームワークの提案

宮﨑 賢一[1,a)]　重野 寛[2,b)]

概要：DDoS 攻撃による被害の深刻化が問題となっており，その攻撃手法も変化するため，未知の攻撃手法に対処し続ける必要がある．しかし，従来の DDoS 攻撃検知手法では，未知の攻撃への対応に大規模再学習を要する点や，処理の複雑性によりリアルタイム検知が困難である点が課題であった．本稿では，未知の攻撃への対応コストおよび検知処理の計算コスト削減を目指し，生パケットベースの畳み込みニューラルネットワークと Meta-Baseline を用いた軽量な DDoS 攻撃検知フレームワークを提案する．メタ学習手法である Meta-Baseline により，未知の攻撃に対して少数サンプルでの対応を実現する．また，各フローのパケットビット列を並べた入力に対して，ビット次元およびパケット次元に独立して 1 次元畳み込みを適用することで，計算コストの抑制と検知性能の両立を実現する．未知の攻撃を含む Few-shot 実験では平均で正解率 0.9670，適合率 0.9702，再現率 0.9670，F1 スコア 0.9666，スループット 30347flows/s を達成し，処理コストを抑えつつ未知の攻撃に対しても高い検知性能をもつことを確認した．

キーワード：DDoS 攻撃，生パケット，ビットレベル処理，CNN，メタ学習

A Lightweight DDoS Attack Detection Framework Using Raw Packet-Based Convolutional Neural Networks and Meta-Learning

Kenichi Miyazaki[1,a)]　Hiroshi Shigeno[2,b)]

Abstract: Distributed Denial of Service (DDoS) attacks cause severe damage and continue to evolve, so it is essential to address unknown attacks continuously. However, conventional DDoS detection methods have limitations: requiring large-scale retraining to adapt to unknown attacks and struggling to achieve real-time detection due to processing complexity. This paper proposes a lightweight DDoS attack detection framework using raw packet-based convolutional neural networks and Meta-Baseline. By leveraging Meta-Baseline, a meta-learning approach, the proposed method can handle unknown attacks with only a few samples. To balance model parameter reduction and detection performance, one-dimensional convolutions along the bit and packet axes are independently applied to an input matrix of packet-bit sequences per flow. In few-shot experiments involving unknown attacks, the proposed method achieved 0.9670 accuracy, 0.9702 precision, 0.9670 recall, 0.9666 F1-score, and throughput 30347 flows/s on average, demonstrating high detection performance for unknown attacks with low processing cost.

Keywords: DDoS attacks, raw packets, bit-level processing, CNN, Meta-Learning

1. はじめに

分散サービス妨害（DDoS: Distributed Denial of Service）攻撃はサイバー空間における重大な脅威となってい

[1] 慶應義塾大学大学院 理工学研究科
Graduate School of Science and Technology, Keio University
[2] 慶應義塾大学 理工学部
Faculty of Science and Technology, Keio University
[a)] miyazaki@mos.ics.keio.ac.jp
[b)] shigeno@mos.ics.keio.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る．今日，サイバー空間は金融や交通，医療などの社会基盤と密接に関係しており，DDoS 攻撃は日常生活にも大きな影響を与える．実際に，交通機関や金融機関をターゲットとした DDoS 攻撃により，大きな被害が発生した [1]．

ネットワーク技術の発展に伴い，DDoS 攻撃の手法も進化しており，高度化し続けている．たとえば，HTTP/2 Rapid Reset 攻撃では HTTP/2 のストリーム多重化を悪用し，ストリームのリクエストとキャンセルの繰り返しによりサービス妨害を行う．また，利用が拡大している QUIC プロトコルでも，その実装に起因する Optimistic ACK 攻撃が報告されている．

DDoS 攻撃に対処するため，侵入検知システム（IDS: Intrusion Detection System）やスクラビングセンタなどのセキュリティソリューションが広く用いられている．IDS では，ネットワークトラフィックに基づき，サイバー攻撃やその他の異常を検知する．スクラビングセンタでは，サイバー攻撃が疑われる場合に，そのターゲット宛の通信を迂回させて引き込み，高度なトラフィック分析等によって悪意のある攻撃トラフィックをフィルタリングし，正常トラフィックのみを本来の宛先に送信する．

これらのセキュリティソリューションのさらなる発展のため，DDoS 攻撃検知に関する研究が積極的に行われている．従来のルールベース手法では，既知の攻撃トラフィックを識別することができるが，未知の攻撃トラフィックについては検知することができない．統計ベース手法 [2] では，IP アドレスのエントロピーなどの統計量を用いて異常検知を行うが，低レート DDoS 攻撃などの一部の攻撃ベクトルに対しては対処が困難である．

近年，機械学習（ML: Machine Learning）の研究が加速し，サイバー攻撃検知にも活用されるようになっている．多くの ML ベース手法では，攻撃検知を，ネットワークフローの統計特徴量に基づく分類問題として定式化し，攻撃検知を行っている [3] [4]．ML ベース手法では，十分なサンプル数が確保できる既知の攻撃に対して，高い精度で検知できることが報告されている．一方で，十分なサンプルが確保できない未知の攻撃に対しては検知が困難である．未知の攻撃への対応を目指し，MAML（Model-Agnostic Meta-Learning） [5] などのメタ学習を用いた少数サンプルでの未知攻撃対応が研究されている [6] [7] [8] [9]．しかし，メタ学習を用いた既存研究の多くでは複雑な処理やモデルが含まれ，処理効率について考慮されていない．

本稿では，未知の攻撃への対応コストとともに検知時の計算コストを削減することを目指し，生パケットベースの畳み込みニューラルネットワーク（CNN）とメタ学習を用いた軽量な DDoS 攻撃検知フレームワークを提案する．提案手法では Meta-Baseline [10] を用いることでシンプルな仕組みでメタ学習を行い，少数サンプルで未知の攻撃に対応する能力を学習する．また，提案するビット・パケット分離畳み込みモデルでは，フローごとの生パケット列を 2 次元配列の入力として直接受け取り，軽量な CNN によりビットレベル処理することで，フロー統計量の計算コストやモデルの複雑性による検知処理の計算コストを削減し，処理の軽量化を図っている．ビット方向とパケット方向に独立して 1 次元畳み込みを適用することでモデルの軽量化と検知性能を両立している．

主な貢献は以下のとおりである：

(1) メタ学習手法である Meta-Baseline を採用した DDoS 攻撃検知フレームワークを提案し，少数サンプルでの未知攻撃対応を実現する．

(2) 提案モデルでは，統計処理等を必要とせずパケットのビット列を直接入力として受け取り，ビット次元とパケット次元をもつ 2 次元の入力に対して各方向に独立に 1 次元畳み込みを適用することで処理軽量化と攻撃検知性能を両立する．

(3) 同一モデルでの複数攻撃クラスに対する検知性能を評価し，提案手法が，複数種類の攻撃が混在する環境において正常トラフィックと攻撃トラフィックを分類できることを示した．

本稿の構成は以下のとおりである．第 2 章では関連研究について述べる．第 3 章で軽量な DDoS 攻撃検知フレームワークを提案する．第 4 章で実験設定について述べた後，第 5 章で実験結果について示す．最後に，第 6 章で結論を述べる．

2. 関連研究

2.1 非 ML ベース手法

Feinstein ら [2] は，パケット特徴量の統計量に基づく DDoS 攻撃検知手法を提案した．IP アドレス等の特定のパケットヘッダフィールドに対して，エントロピーやカイ二乗値を計算し，あらかじめ設定した閾値と比較することで DDoS 攻撃を検知する．しかし，統計量比較に基づく手法では，攻撃やバックグラウンドトラフィックの特徴変化に対して頻繁な閾値調整が必要であることや，統計量の変化が小さい低レート DDoS 攻撃などの検知が困難であることが課題である．

2.2 ML ベース手法

ML の発展に伴い，サイバー攻撃検知にも ML が活用されるようになった．Yuan ら [3] は再帰ニューラルネットワーク（RNN: Recurrent Neural Network）と CNN を用いた DDoS 攻撃検知手法 DeepDefense を提案した．DeepDefense はパケット列を時系列データとして RNN で処理することで最も直近のパケットが正常か攻撃かを判別する．

© 2026 Information Processing Society of Japan

Doriguzzi-Corin ら [4] は軽量な深層学習 DDoS 検知システム LUCID を提案した．LUCID ではパケット長や IP フラグなどのパケット特徴量を軽量な CNN で処理することで DDoS 攻撃を検知する．

これらの検知手法では，ML を用いることで専門知識に基づいて計算された統計量ではなく，パケットヘッダから得ることができる生パケットデータを用いて検知を行っている．十分な数のサンプルを用いて大規模学習を行った既知の攻撃ベクトルに対しては高い精度で検知できることが報告されているが，十分なサンプルが確保できない未知の攻撃への対応は考慮されていない．また，これらは，正常と攻撃の 2 値判定を行うため，複数の攻撃ベクトルが存在する環境では検知性能が不明である．

2.3 Few-shot 学習ベース手法

少数サンプルで未知の攻撃ベクトルに対応するため，Few-shot 学習の手法を導入した攻撃検知が研究されている．Xu ら [6] はパケットバイト列に基づき，サンプル比較によりネットワークトラフィックの攻撃判定を行う，メタ学習ベースのネットワーク侵入検知手法 FC-Net を提案した．FC-Net では，推論対象のサンプルとラベル付の既知サンプルに対して，CNN を用いた特徴抽出ネットワーク F-Net により特徴抽出を行い，比較ネットワーク C-Net でその特徴量をサンプルどうしで比較することで推論対象のネットワークフローを判定する．FC-Net ではネットワークフローごとに比較を行うため，比較の基準となるサンプルを確保すれば，大規模な再学習を必要とせず，未知の攻撃にも Few-shot（少数サンプル）で対応することができる．一方で，推論ごとに複数サンプルとの比較が必要となるため計算量が多くなりやすく，リアルタイム検知に限界がある．

Yu ら [7] はパケットバイトを CNN で階層的に処理する PBCNN を提案した．PBCNN では，パケットバイトを 256 次元に One-hot エンコーディングし，バイト間の関係を抽出する CNN とパケット間の関係を抽出する CNN を階層的に処理することで攻撃検知を行う．また，Siamese Network を使い，Few-shot での攻撃検知を考慮している．しかし，モデルや推論手順の複雑性から，リアルタイム検知には課題がある．

Barut ら [8] はパケットヘッダの生データを Transformer モデルで処理する悪性トラフィック分類システム R1DIT（Residual 1-D Image Transformer）を提案した．R1DIT では，ネットワークプロトコルに応じたパケットヘッダ前処理と，特徴量埋め込みおよび位置エンコーディング，さらに Transoformer モデルを組み合わせることで，ネットワークフローの多クラス分類を実現している．また，転移学習や Few-shot 学習を用いることで，Few-shot で未知の攻撃ベクトルに対応することを可能としている．

Alfatemi ら [9] は Few-shot DDoS 攻撃検知のための，Prototypical Network [11] と MAML [5] を組み合わせたメタ学習フレームワーク ProtoMAML を提案した．ProtoMAML では，ネットワークフロー統計量を入力として，埋め込み関数に基づき距離ベース分類を行う Prototypical Network を，MAML によるタスクベース学習で高速適応することで，Few-shot での未知攻撃対応を実現している．しかし，ProtoMAML では正常と攻撃の 2 値分類を前提として実装されており，複数攻撃ベクトル混在環境における検知性能は不明である．

3. 提案手法

本稿では，未知の攻撃への対応コストと検知処理の計算コストの削減を目的とし，生パケットベースの CNN と Meta-Baseline を用いた軽量な DDoS 攻撃検知フレームワークを提案する．提案手法は生パケットを入力として攻撃判定結果を出力することで，エンドツーエンドの攻撃検知を行う．パケットビット列を用いるため，IP や TCP/UDP 以外のパケット解析や DPI（Deep Packet Inspection）を必要とせず，また，高度な専門知識なしで未知のネットワークプロトコルにも対応できる．提案モデルでは，フローごとにパケットビット列を並べた 2 次元の入力に対して，各次元に独立して 1 次元畳み込みを適用することで，計算効率と検知性能を両立させる．さらに，メタ学習手法の 1 つである Meta-Baseline [10] を用いることで，モデルは少数サンプルから攻撃のトラフィックパターンを学習する．

はじめに，キャプチャしたパケットをフローごとのパケット列に分割し，ビット次元とパケット次元をもつ 2 次元配列として構成する．次に，ビット・パケット分離畳み込みモデルにより，2 次元配列から埋め込み特徴量が計算される．最後に，その埋め込み特徴量を，あらかじめ計算したトラフィッククラスごとの平均埋め込み特徴量と比較することで攻撃フローを検知する．**表 1** に記号一覧を示す．

3.1 ネットワークトラフィック前処理

被害ターゲット宛に送られたパケットは t [s] ごとに通信路上で順にキャプチャされる．ここで，キャプチャされるパケットには，被害ターゲット宛のものが含まれ，被害ターゲットから送信されたものは含まれない．キャプチャされたパケットはタイムウィンドウ内でフローごとにグループ化した後，一部のフィールドがマスクされ，さらにモデル入力用に 2 次元配列に整形される．**図 1** にネットワークトラフィック前処理の流れを示す．

グループ化

キャプチャされたパケットは 5 タプル $(src_ip, src_port, dest_ip, dest_port, protocol)$ に基づ

© 2026 Information Processing Society of Japan

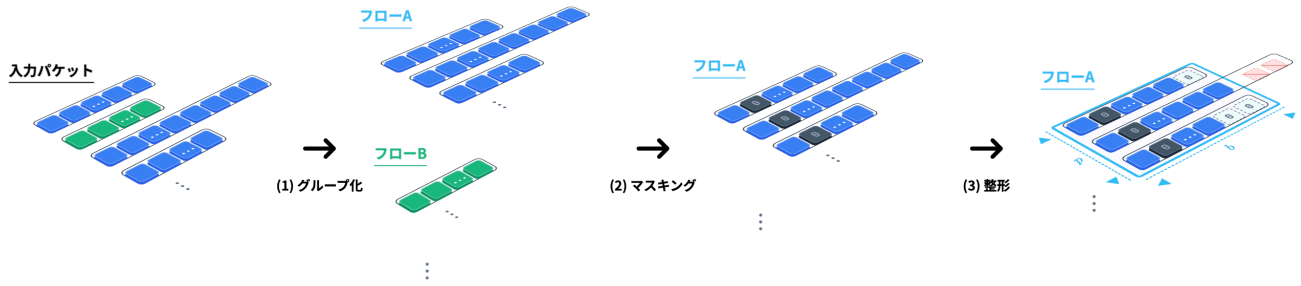


図 1　ネットワークトラフィック前処理
Fig. 1　Network traffic preprocessing

表 1　記号一覧
Table 1　List of symbols

記号	定義
t	フロー分割のタイムウィンドウ長（s）
p	フローの最大パケット数
b	最大パケット長（bit）
j_{bit}	ビット次元畳み込みのカーネルサイズ
s_{bit}	ビット次元畳み込みのストライド
j_{pkt}	パケット次元畳み込みのカーネルサイズ
s_{pkt}	パケット次元畳み込みのストライド
f_{out}	モデルの出力次元数
c	トラフィッククラス
$\mathbf{e}$	埋め込み特徴量
$\mathbf{e}_c$	クラス c の平均埋め込み特徴量
n	メタ訓練タスクのトラフィッククラス数
k_{train}	サポートセット内，各クラスのフローサンプル数
q_{train}	クエリセット内，各クラスのフローサンプル数
k_{test}	各平均埋め込み特徴量計算のフローサンプル数

いてフローごとにグループ化される．ここで，*src_ip* は送信元 IP アドレス，*src_port* は送信元ポート番号，*dest_ip* は宛先 IP アドレス，*dest_port* は宛先ポート番号，*protocol* は IP ヘッダのプロトコル番号である．これらの 5 要素のいずれかが欠けているパケット，すなわち，IP もしくは TCP/UDP を使っていないパケットは対象外とする．

マスキング

パケットの一部フィールドをマスクし，匿名化する．各パケットの *src_ip*, *src_port*, *dest_ip*, *dest_port* の部分はゼロでマスクされる．これにより，モデルは，IP アドレスやポート番号以外のパケットフィールドから DDoS 攻撃を識別する能力を学習する．

整形

マスキングに続いて，各パケットグループは定型の 2 次元配列に整形される．t [s] のタイムウィンドウ内で，各フローについて p パケットを並列に連結する．さらに，各パケットについて IP ヘッダ以降 b [bit] を抽出する．パケット数が p に満たない場合は，空いている部分をゼロ埋めする．同様に，IP ヘッダ以降のパケット長が b [bit] に満たない場合は，空いている部分をゼロ埋めする．結果として，ビット次元とパケット次元をもつ $p \times b$ のビット配列が生成される．ここで，ビット次元は各パケットのビットの並び（サイズ b）を表す軸で，パケット次元はフロー内のパケットの並び（サイズ p）を表す軸である．

3.2　モデル構造

本稿で提案するビット・パケット分離畳み込みモデルは，フローごとの生パケット列を入力として受け取り，その埋め込み特徴量を出力する特徴抽出器として機能する．提案モデルは，入力層，ビット次元畳み込み層，パケット次元畳み込み層，グローバル平均プーリング（GAP: Global Average Pooling）層，全結合層から構成される．

提案モデルのモデル構造を**図 2** に示す．モデルは，ビット次元とパケット次元をもつ 2 次元配列を入力として受け取る．まず，入力に対してビット次元に沿った 1 次元畳み込みを適用する．ビット次元方向の畳み込みではカーネルサイズ j_{bit} およびストライド s_{bit} を 1 B あたりのビット数，すなわち 8 に設定する．パケットフィールドの多くはバイト単位で意味をもつ一方，その他のパケットフィールドではビット単位で意味をもつフラグ等も含まれる．そのため，バイト単位のスカラ表現（0–255）として処理するのではなく，バイト（8 bit）ごとに畳み込みフィルタでビットレベル処理する．これにより，パケットフィールドのバイト境界構造を維持したままビット表現を低次元特徴量に圧縮し，後続処理の計算量を大幅に削減する．次に，ビット次元畳み込み層の出力に対してパケット次元に沿った 1 次元畳み込みを適用する．パケット方向の畳み込みでは，カーネルサイズ j_{pkt} とストライド s_{pkt} に基づいて畳み込みフィルタを適用し，フローごとのトラフィックパターンを捉える．畳み込み処理された特徴量は GAP 層でプーリング処理され，最後に，全結合層により f_{out} 次元の埋め込み特徴量 $\mathbf{e}$ が出力される．

3.3　学習・推論手順

提案フレームワークでは，限られたサンプルで未知の攻

© 2026 Information Processing Society of Japan

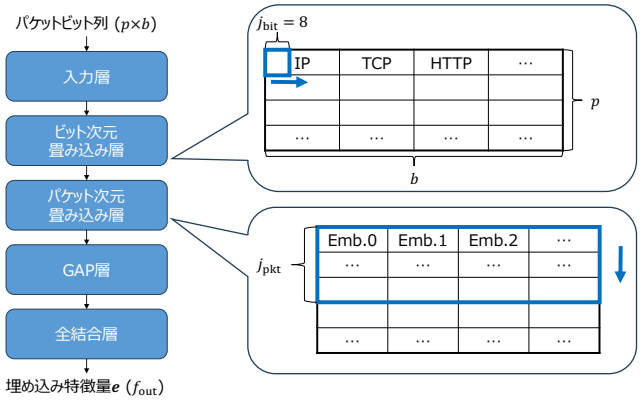


図 2　モデル構造
Fig. 2　Model architecture

Algorithm 1 Meta-training

Input: Feature extractor f_θ, Training dataset $\mathcal{D}$, # of steps S_{meta}, Batch size B_{meta}, Learning rate η_{meta}, # of classes in a task n, Temperature τ
Output: Trained feature extractor weights θ^*

for step $= 1$ **to** S_{meta} **do**
  $\mathcal{L}_{batch} \leftarrow 0$
  for $b = 1$ **to** B_{meta} **do**
    Sample task $T \leftarrow ((X_S, Y_S), (X_Q, Y_Q))$ from $\mathcal{D}$
    for each class c in Y_S **do**
      $\mathbf{e}_c \leftarrow \frac{1}{|X_S^c|} \sum_{x_i \in X_S^c} f_\theta(\mathbf{x}_i)$ {X_S^c: Subset of X_S belonging to c}
    end for
    $\mathcal{E} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n]$
    $E_Q \leftarrow f_\theta(X_Q)$
    $Z \leftarrow \text{CosineSimilarity}(E_Q, \mathcal{E}) \times \tau$
    $\mathcal{L}_{task} \leftarrow \text{CrossEntropy}(Y_Q, Z)$
    $\mathcal{L}_{batch} \leftarrow \mathcal{L}_{batch} + \mathcal{L}_{task}$
  end for
  $\mathcal{L}_{batch} \leftarrow \frac{1}{B_{meta}} \mathcal{L}_{batch}$
  $\theta \leftarrow \text{Optimize}(\mathcal{L}_{batch}, \eta_{meta})$
end for
return $\theta^* \leftarrow \theta$

撃手法に対応するため，メタ学習手法である Meta-Baseline [10] に基づいて学習を行う．Meta-Baseline に基づき，学習はベース訓練とメタ訓練の 2 段階で行われる．ベース訓練では，特徴抽出器に分類ヘッドが付加され，一般的な多クラス分類のモデル訓練が行われる．メタ訓練では，ベース訓練時に付加された分類ヘッドは除去され，ベース訓練を経た特徴抽出器に対してタスクベースのメタ学習を行う．各タスクは n-way k_{train}-shot のサポートセットと n-way q_{train}-shot のクエリセットで構成され，n 種類のトラフィッククラス（正常クラスと $n-1$ 種類の攻撃クラス）に対して各クラス k_{train} および q_{train} フローずつ，合計で $n \cdot (k_{\text{train}} + q_{\text{train}})$ フローサンプルが含まれる．タスクに含まれるトラフィッククラスやフローサンプルを変えながら，多くのタスクを学習することで，少数サンプルからトラフィッククラスを識別する能力を獲得する．メタ訓練の詳細（バリデーションは省略）を Algorithm 1 に示す．

Algorithm 2 Inference

Input: Trained feature extractor f_θ, Average embeddings $\mathcal{E} = \{\mathbf{e}_c \mid c \in \mathcal{C}, \mathcal{C}$: All traffic classes$\}$, Data for inference $\mathbf{x}$
Output: Predicted class $\hat{y}$

$\mathbf{e} \leftarrow f_\theta(\mathbf{x})$
$CS \leftarrow \text{CosineSimilarity}(\mathbf{e}, \mathcal{E})$
$\hat{y} \leftarrow \arg\max_{c \in \mathcal{C}}(CS)$
return $\hat{y}$

訓練した特徴抽出器を用いて，トラフィッククラスごとに k_{test} フローサンプルから平均埋め込み特徴量を計算し，データベースに保存する．推論時には同様に，特徴抽出器により各フローの埋め込み特徴量 $\mathbf{e}$ を計算し，式 (1) のコサイン類似度（CS）に基づいて，データベースに保存されている各トラフィッククラスの平均埋め込み特徴量 $\mathbf{e}_c$（c はトラフィッククラス）と比較する．

$$CS = \cos(\mathbf{e}, \mathbf{e}_c) = \frac{\mathbf{e} \cdot \mathbf{e}_c}{\|\mathbf{e}\| \|\mathbf{e}_c\|} \quad (1)$$

コサイン類似度は-1 から 1 の範囲の値を取り，値が大きいほど特徴量空間上での向きが近いことを示す．したがって，各フローは，コサイン類似度の最も高いトラフィッククラスに属すると予測され，そのクラスによって攻撃であると判定される．推論手順を Algorithm 2 に示す．

4. 実験設定

提案手法の Few-shot 性能を評価するため実験を行った．本節では，データセットや評価指標，モデル実装，実験環境について述べる．

4.1 データセット

実験では，ネットワークセキュリティの研究で広く利用されている CIC-DDoS2019 データセット [12] を用いて提案手法を評価した．提案モデルはフローごとのパケットビット列を入力として受け取るため，CIC-DDoS2019 データセットのサンプルを適切なフォーマットに再構成して用いた．具体的には，CIC-DDoS2019 データセットの PCAP ファイルに含まれるパケットデータを t [s] のタイムウィンドウごとに分割し，5 タプル $(src_ip, src_port, dest_ip, dest_port, protocol)$ に基づいてフローごとにグループ化した．続いて，各グループをトラフィッククラスでラベル付した．

CIC-DDoS2019 データセットは，テストベッドで DDoS 攻撃を再現した 2 日間のネットワークトラフィックで構成されている．12 クラスの攻撃トラフィックを含む「01-12」（1 月 12 日）と 7 クラスの攻撃トラフィックを含む「03-11」（3 月 11 日）があり，いずれにも攻撃トラフィックに加えて現実的なバックグラウンドの正常トラフィックが含まれている．**表 2** に示すとおり，「01-12」サブセットには「03-11」サブセットと同じ攻撃クラス（PortMap を除く）に加え，他 6 クラスの攻撃が含まれている．実験では，「03-11」サ

© 2026 Information Processing Society of Japan

表 2　CIC-DDoS2019 データセットの攻撃クラス
Table 2　Attack classes in CIC-DDoS2019 dataset

サブセット	攻撃
03-11	PortScan, NetBIOS*, LDAP*, MSSQL*, UDP*, UDP-Lag*, SYN*
01-12	NetBIOS*, LDAP*, MSSQL*, UDP*, UDP-Lag*, SYN*, NTP, DNS, SNMP, SSDP, WebDDoS, TFTP

*は共通

ブセットに含まれる 7 クラスの攻撃サンプルと正常サンプルでベース訓練およびメタ訓練，「01-12」サブセットに含まれる 12 クラスの攻撃サンプルと正常サンプルで各クラスの平均埋め込み特徴量計算とテストを行い，Few-shot での検知性能を評価した．

4.2　評価指標

本稿ではモデルの検知性能指標として，正常クラスと 1 つの攻撃クラスの 2 値分類タスクに対する正解率 Acc，適合率 Pre，再現率（検知率）Rec，F1 スコア $F1$ を報告する．各指標は，真陽性 TP，真陰性 TN，偽陽性 FP，偽陰性 FN として，式 (2)–(5) のとおり計算される．

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

$$Pre = \frac{TP}{TP + FP} \tag{3}$$

$$Rec = \frac{TP}{TP + FN} \tag{4}$$

$$F1 = \frac{2 \cdot Pre \cdot Rec}{Pre + Rec} \tag{5}$$

さらに，処理効率の指標として，モデルのパラメータ数と，1 s あたりの処理フロー数を表すスループット（flows/s）も評価する．

4.3　モデル実装

提案モデルは入力層，2 つの畳み込み層，GAP 層，全結合層で構成される．入力層では，$b = 480\text{bit} = 60\text{B}$ として $p \times b$ の 2 次元配列を受け取る．ビット次元畳み込み層は，フィルタ数 32，カーネルサイズ $j_{\text{bit}} = 8$，ストライド $s_{\text{bit}} = 8$，パディングなしである．パケット次元畳み込み層は，フィルタ数 32，カーネルサイズ $j_{\text{pkt}} = 3$，ストライド $s_{\text{pkt}} = 1$，パディングあり（ゼロパディング）である．また，これら 2 つの畳み込み層の後にそれぞれ ReLU を適用している．最後の全結合層では，$f_{out} = 32$ 次元の埋め込み特徴量を出力する．モデルのベース訓練およびメタ訓練では，最適化アルゴリズムとして Adam [13] を用いた．

本実験におけるハイパーパラメータは，網羅的なハイパーパラメータ最適化に先立ち，関連研究等に基づいて設定した．そのうち，ベース訓練の学習率 η_{base} は，Adam [13] に基づいて設定した．メタ訓練の温度（スケーリング係数）τ とメタ訓練のバッチサイズ B_{meta} は，Meta-Baseline [10] に基づいて設定した．メタ訓練の学習率 η_{meta} は，η_{base} の設定値 [13] と，Meta-Baseline [10] における η_{base} に対する η_{meta} の倍率に基づいて設定した．ベース訓練のバッチサイズ B_{base} は R1DIT [8] に基づいて設定した．

表 3　実験環境
Table 3　Experimental environment

OS	Ubuntu 22.04 LTS
CPU	Intel Xeon Bronze 3104 CPU @ 1.70GHz
RAM	32GB
GPU	NVIDIA Quadro P620
VRAM	2GB
Nvidia-driver	580.142
CUDA	11.3
cuDNN	8.2.1
Python	3.10.13
Numpy	1.26.4
Tensorflow	2.8.2

4.4　実験環境

本実験は，Intel(R) Xeon(R) Bronze 3104 CPU @ 1.70GHz と NVIDIA Quadro P620 GPU を使い，Ubuntu 22.04 LTS 上で行った．その他の実験環境は**表 3** に示すとおりである．

5.　実験結果

本節では実験の評価結果を示す．結果として，攻撃クラス平均で最高 0.9666 の F1 スコアを達成し，同モデルのスループットは 30347 flows/s であった（$p = 5$，$k_{\text{test}} = 10$）．

5.1　検知性能

表 4 と**表 5** に検知性能の実験結果を示す．これらの値は，正常クラスと 1 つの攻撃クラスからなる 2 値分類タスクに対する指標を全クラス（「01-12」サブセットの 12 クラス）で平均した結果である．

表 4 は，各フローのパケット数 p を 1, 5, 10 に設定した場合の攻撃検知性能である．$k_{\text{test}} = 5$ において，提案手法は $p = 1$ で最高 0.9509 の F1 スコアを達成した．表 4 より，いずれの評価指標についても，$p = 1, 5$ の場合が $p = 10$ の場合より高い結果となった．

従来の発想では，フローに対する情報量の多い $p = 10$ の場合の方が高い性能をもつと考えられるが，実験結果では逆の傾向が見られた．これは，GAP 層を含む提案モデルの構造によるものと考えられる．提案モデルでは 2 層の畳み込み層を経た埋め込み表現は，GAP 層により平均化さ

© 2026 Information Processing Society of Japan

表 4　検知性能 ($p = \{1, 5, 10\}, k_{\text{test}} = 5$)
Table 4　Detection performance ($p = \{1, 5, 10\}, k_{\text{test}} = 5$)

p	正解率	適合率	再現率	F1 スコア
1	0.9520	0.9578	0.9520	0.9509
5	0.9513	0.9557	0.9513	0.9508
10	0.8202	0.8580	0.8202	0.8158

表 5　検知性能 ($p = 5, k_{\text{test}} = \{1, 5, 10\}$)
Table 5　Detection performance ($p = 5, k_{\text{test}} = \{1, 5, 10\}$)

k_{test}	正解率	適合率	再現率	F1 スコア
1	0.9373	0.9432	0.9373	0.9366
5	0.9513	0.9557	0.9513	0.9508
10	0.9670	0.9702	0.9670	0.9666

れる．一方で，入力は固定サイズの 2 次元配列であり，パケット数やパケット長が不足している場合にはゼロパディングされるため，値の大きい $p = 10$ の方が，タイムウィンドウ内の各フローに実際に存在するパケット数が不足しやすく，疎な配列となりやすい．疎な配列が入力されると，GAP 層の平均化によって畳み込み層で抽出した有意なパケットの埋め込み表現が弱くなるため，$p = 1, 5$ の場合と比較して性能が低くなったと考えられる．これは，別の視点で捉えると，提案モデルは少数のパケットビット列から攻撃の特徴を抽出できるといえる．ただし，この原因の正確な検証にはさらなる実験が必要である．

表 5 は，各クラスの平均埋め込み特徴量の計算に使うラベル付サンプル数 k_{test} を 1, 5, 10 に設定した場合の攻撃検知性能である．$p = 5$ においては，$k_{\text{test}} = 10$ で最高 0.9666 の F1 スコアを達成した．表 5 では，k_{test} が大きいほど検知性能も高い傾向が見られるが，これは，より多くのサンプルから安定して埋め込み特徴量を計算することでサンプル間の偏りが平均埋め込み特徴量に反映されにくく，網羅的に攻撃の特徴を捉えることができるためだと考えられる．

5.2　推論効率

表 6 に $k_{\text{test}} = 5$ に対して $p = 1, 5, 10$ の場合の推論効率に関する評価結果を示す．これらのスループットは，ネットワークトラフィック前処理を含まないモデル推論時間で，全攻撃クラスの平均である．実験では，$p = 1$ で最高スループット 38436 flows/s を達成した．p が小さいほどスループットが高い傾向が見られるが，これは，1 フローに対する計算量が，フローごとに計算対象とするパケット数 p に大きく依存するためである．

表 7 に $p = 5$ に対して $k_{\text{test}} = 1, 5, 10$ の場合の処理効率に関する評価結果を示す．k_{test} の変化に対しては，各サンプルに対する推論処理が変わらないため，いずれの場合もおよそ同等のスループットとなっている．

表 6　推論効率 ($p = \{1, 5, 10\}, k_{\text{test}} = 5$)
Table 6　Inference efficiency ($p = \{1, 5, 10\}, k_{\text{test}} = 5$)

p	スループット（flows/s）
1	38436
5	30366
10	19570

表 7　推論効率 ($p = 5, k_{\text{test}} = \{1, 5, 10\}$)
Table 7　Inferece efficiency ($p = 5, k_{\text{test}} = \{1, 5, 10\}$)

k_{test}	スループット（flows/s）
1	27836
5	30366
10	30347

表 8　比較評価
Table 8　Comparison

手法	再現率	スループット（flows/s）
R1DIT [8] ($p = 2, b = 50, k_{\text{test}} = 10$)	0.9892	5737
提案手法 ($p = 5, b = 60, k_{\text{test}} = 10$)	0.9666	30347

5.3　比較評価

表 8 に 10-shot における R1DIT [8] との比較評価を示す．ただし，これらの結果は R1DIT の論文 [8] で報告された値（スループットは論文記載の値をもとに算出）であり，厳密に実験条件や実験環境を合わせて評価した結果でないことに注意する．表 8 より，攻撃検知率（再現率）について，R1DIT が 0.0226 上回るものの，大きな差はない．一方，スループットは，提案手法が R1DIT の約 5.3 倍である 30347 flows/s を達成し，提案手法が大きく上回る．これらの比較結果より，提案手法は小さい計算コストで同等の検知性能を達成しているといえる．

6.　おわりに

本稿では，未知の攻撃への対応コストと検知処理の計算コストの削減を目的として，軽量な CNN とメタ学習手法である Meta-Baseline を用いた DDoS 攻撃検知フレームワークを提案した．ネットワークフローごとのパケットビット列に対して，ビット次元とパケット次元に独立した畳み込み処理を順に行うことで，計算量を抑えつつ，パケットビットが表す詳細な情報を捉える．また，Meta-Baseline により，既存のネットワークトラフィックデータを十分に活かしてモデルを学習し，未知の攻撃に対しても少数サンプルでの対応を可能とする．標準的なデータセットを用いた実験では，全攻撃クラス平均で正解率 0.9670，適合率 0.9702，再現率 0.9670，F1 スコア 0.9666，スループット 30347flows/s を達成した．今後の研究では，モデル構造の

© 2026 Information Processing Society of Japan

見直しによる性能向上や，多クラス分類タスクに対する検知性能，ネットワークトラフィック前処理を含めた全体のリアルタイム検知性能についても明らかにしていきたい．

謝辞 第一著者は公益財団法人 里見奨学会から奨学金の支援を受けています．ここに深謝の意を表します．

参考文献

[1] 独立行政法人情報処理推進機構：情報セキュリティ 10 大脅威 2025 組織編，報告書，独立行政法人情報処理推進機構 (2025). Accessed: 2026-08-20.

[2] Feinstein, L., Schnackenberg, D., Balupari, R. and Kindred, D.: Statistical approaches to DDoS attack detection and response, *Proceedings DARPA Information Survivability Conference and Exposition*, Vol. 1, pp. 303–314 vol.1 (online), DOI: 10.1109/DISCEX.2003.1194894 (2003).

[3] Yuan, X., Li, C. and Li, X.: DeepDefense: Identifying DDoS Attack via Deep Learning, *2017 IEEE International Conference on Smart Computing (SMARTCOMP)*, pp. 1–8 (online), DOI: 10.1109/SMARTCOMP.2017.7946998 (2017).

[4] Doriguzzi-Corin, R., Millar, S., Scott-Hayward, S., Martínez-del Rincón, J. and Siracusa, D.: Lucid: A Practical, Lightweight Deep Learning Solution for DDoS Attack Detection, *IEEE Transactions on Network and Service Management*, Vol. 17, No. 2, pp. 876–889 (online), DOI: 10.1109/TNSM.2020.2971776 (2020).

[5] Finn, C., Abbeel, P. and Levine, S.: Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks, *arXiv*, (online), available from ⟨https://arxiv.org/abs/1703.03400⟩ (2017). Accessed: 2026-08-20.

[6] Xu, C., Shen, J. and Du, X.: A Method of Few-Shot Network Intrusion Detection Based on Meta-Learning Framework, *IEEE Transactions on Information Forensics and Security*, Vol. 15, pp. 3540–3552 (online), DOI: 10.1109/TIFS.2020.2991876 (2020).

[7] Yu, L., Dong, J., Chen, L., Li, M., Xu, B., Li, Z., Qiao, L., Liu, L., Zhao, B. and Zhang, C.: PBCNN: Packet Bytes-based Convolutional Neural Network for Network Intrusion Detection, *Computer Networks*, Vol. 194, p. 108117 (online), DOI: https://doi.org/10.1016/j.comnet.2021.108117 (2021). Accessed: 2026-08-20.

[8] Barut, O., Luo, Y., Li, P. and Zhang, T.: R1DIT: Privacy-Preserving Malware Traffic Classification With Attention-Based Neural Networks, *IEEE Transactions on Network and Service Management*, Vol. 20, No. 2, pp. 2071–2085 (online), DOI: 10.1109/TNSM.2022.3211254 (2023).

[9] Alfatemi, A., Rahouti, M., Bhuiyan, Z. A., Soyman, A. and Aledhari, M.: ProtoMAML: A Hybrid Meta-Learning Approach Integrating Prototypical Networks for Data-Efficient DDoS Attack Detection, *2025 International Wireless Communications and Mobile Computing (IWCMC)*, pp. 1144–1149 (online), DOI: 10.1109/IWCMC65282.2025.11059701 (2025).

[10] Chen, Y., Liu, Z., Xu, H., Darrell, T. and Wang, X.: Meta-Baseline: Exploring Simple Meta-Learning for Few-Shot Learning, *arXiv*, (online), available from ⟨https://arxiv.org/abs/2003.04390⟩ (2021). Accessed: 2026-08-20.

[11] Snell, J., Swersky, K. and Zemel, R. S.: Prototypical Networks for Few-shot Learning, *arXiv*, (online), available from ⟨https://arxiv.org/abs/1703.05175⟩ (2017). Accessed: 2026-08-20.

[12] Sharafaldin, I., Lashkari, A. H., Hakak, S. and Ghorbani, A. A.: Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy, *2019 International Carnahan Conference on Security Technology (ICCST)*, pp. 1–8 (online), DOI: 10.1109/CCST.2019.8888419 (2019).

[13] Kingma, D. P. and Ba, J.: Adam: A Method for Stochastic Optimization, *arXiv*, (online), available from ⟨https://arxiv.org/abs/1412.6980⟩ (2017). Accessed: 2026-08-20.

© 2026 Information Processing Society of Japan